

FULL STACK AI ENGINEER INTERN PROJECT SUBMISSION

AI Prompts Used During Development

A curated compilation of senior-level, production-grade engineering prompts formulated across the full-stack AI lifecycle — establishing architecture, backend security, RAG pipeline mechanics, database vector indexes, and cloud deployments.

DEVELOPER	PROJECT	PRODUCTION URL	REPOSITORY
Sathwik Bodakunta	AiProf Study Companion	ai-prof-study-companion.vercel.app	github.com/sathwik1821/Ai-Prof-Study-Companion

Engineering Specification & Prompt Methodology Standard

This document presents the **curated collection of professional engineering prompts required to design, implement, secure, and deploy the AiProf platform**. In production AI-assisted software engineering, senior developers do not treat AI as a casual search engine; they drive autonomous AI coding agents with rigorous architectural constraints, explicit technology choices, strict security requirements, and verifiable acceptance criteria.

Every prompt documented herein details the precise engineering directive, technical objective, AI output contribution, and developer validation procedure across 10 essential software domains.

Security & Redaction Standard: All production credentials, database connection secrets, API tokens, and personal email addresses have been securely sanitized using [REDACTED] .

Development Domain Coverage Matrix

DOMAIN	PROMPTS	KEY ENGINEERING SCOPE	TECHNOLOGIES IMPACTED
A. Architecture & System Design	Prompts 01–02	3-tier cloud topology & evaluation rubric feature audit	Vercel, Render, Neon, Spring Boot, React 19
B. Frontend Engineering	Prompts 03–05	Space/Project metadata editing, sidebar UX, Targeted Mastery cards	React 19, Lucide Icons, Vanilla CSS Grid
C. Backend Engineering	Prompts 06–07	Role-based access control (RBAC: USER vs ADMIN), admin telemetry guards	Spring Security 6.4, JWT Filter, Spring Data JPA
D. Database & Data Layer	Prompts 08–09	Neon PostgreSQL 16 setup, pgvector 3072d index, cascade deletion SQL	Neon Serverless, pgvector, Flyway, Hibernate
E. AI / LLM Engineering	Prompts 10–11	Dynamic concept mastery algorithm, Empty Materials RAG guard gate	Gemini Flash Lite, Gemini Embeddings, pgvector
F. Authentication & Security	Prompts 12–13	Google OAuth 2.0 Identity Services, 6-digit email OTP verification gate	Google ID Token Verifier, Resend API, BCrypt
G. Debugging & Problem Solving	Prompts 14–16	OAuth Error 400 origin mismatch, SMTP port 587 block, React Error #310	Google Cloud Console, Resend REST, React Hooks
H. Testing & Validation	Prompts 17–18	End-to-end vector pipeline validation, live production DOM browser audit	Chrome Headless, JUnit 5, Browser Subagent
I. Deployment & Configuration	Prompts 19–20	Production OAuth origin sync, Vercel Deployment Protection bypass	Vercel CLI, Render Docker, GitHub CI/CD
J. Documentation	Prompts 21–23	Public submission README, Architecture PDF, AI Tools & Usage PDF	Markdown, Headless Chrome Engine, Node.js

A. Architecture & System Design

ARCHITECTURE

Prompt 01 — Three-Tier Decoupled Cloud System Topology Design

SYSTEM ARCHITECTURE

"Architect a modern, decoupled three-tier cloud deployment topology for 'AiProf', an AI-powered personal study companion. The system requires a single-page application frontend, a containerized RESTful API backend, a managed relational database supporting high-dimensional vector search for RAG workflows, and external LLM inference.

Constraints & Specifications:

- 1. Frontend: React 19 SPA built with Vite, deployed to Vercel edge networks for global static distribution.
- 2. Backend: Java 21 and Spring Boot 3.4, packaged as a multi-stage Docker container deployed to Render.
- 3. Database: Serverless PostgreSQL 16 on Neon, using pooled connections with mandatory SSL encryption and the pgvector extension.
- 4. AI Inference: Google Gemini Flash Lite for reasoning and gemini-embedding-001 for 3072-dimensional vector embeddings.

Provide the complete system architecture diagram, inter-tier communication protocols over TLS, and an incremental zero-downtime deployment plan."

ENGINEERING OBJECTIVE:	Establish an enterprise-grade decoupled cloud topology separating static asset delivery, backend compute, vector storage, and LLM inference.
AI CONTRIBUTION:	Designed the three-tier architecture diagram, formulated multi-stage Dockerfile configurations, structured CORS policies, and established cloud deployment sequences.
DEVELOPER CONTRIBUTION:	Evaluated cloud hosting tiers, provisioned Vercel, Render, and Neon workspaces, linked GitHub CI/CD webhooks, and validated container spin-up.

Prompt 02 — Comprehensive Gap Analysis & Evaluation Rubric Compliance Audit

EVALUATION AUDIT

"Perform a rigorous technical gap analysis of the AiProf platform against full-stack AI engineering evaluation standards before freezing the codebase for submission.

Audit Areas:

- 1. RAG Vector Pipeline: Vector similarity search latency, citation grounding accuracy, hallucination prevention, and prompt token efficiency.
- 2. Authentication & Authorization: Google OAuth 2.0 implementation, email OTP verification, JWT signature validity, and strict RBAC enforcement.
- 3. Reliability & Observability: Spring Security filter chain order, rate-limiting, and administrative telemetry endpoints.

Identify any remaining vulnerabilities or architectural gaps, classify by severity, and propose concrete code changes."

ENGINEERING OBJECTIVE:	Conduct a formal architectural gap analysis comparing implemented application features against the internship evaluation criteria before freezing the codebase.
AI CONTRIBUTION:	Systematically audited repository modules against rubric requirements, identifying that while core RAG features were complete, Google OAuth and email OTP authentication needed strengthening before final documentation.
DEVELOPER CONTRIBUTION:	Prioritized Google OAuth 2.0 and email OTP security hardening over minor UI cosmetics and explicitly enforced a code-freeze guardrail during documentation.

B. Frontend Engineering

FRONTEND

Prompt 03 — Space & Project Metadata Management with Optimistic UI

UI / STATE MANAGEMENT

"Implement a responsive modal management interface in React 19 allowing students to update space and project metadata (title, description, and color tags) directly in `WorkspaceView.jsx` and `ProjectView.jsx`. Requirements: Ensure optimistic UI updates with automatic rollback on API failure, implement client-side input validation, and synchronize changes with the global navigation sidebar without requiring a page reload."

ENGINEERING OBJECTIVE:	Enable students to dynamically edit and update names and descriptions for workspaces and study projects with instant optimistic UI feedback.
AI CONTRIBUTION:	Implemented modal dialog components in <code>WorkspaceView.jsx</code> and <code>ProjectView.jsx</code> , wired PUT API mutation handlers, and handled form validation.
DEVELOPER CONTRIBUTION:	Tested inline name updating across multiple test spaces, validated form input boundaries, and confirmed state synchronization across sidebar navigation.

Prompt 04 — Ergonomic Navigation Sidebar Refactoring & Account Popover

NAVIGATION ERGONOMICS

"Refactor the primary navigation sidebar (`Sidebar.jsx`) to maximize vertical space for academic spaces and active study projects. Requirements: Eliminate the standalone 'Profile' tab, move user account settings and logout into an interactive popover card anchored to the bottom user avatar drawer, and ensure smooth outside-click dismissal and mobile drawer responsiveness."

ENGINEERING OBJECTIVE:	Streamline sidebar navigation ergonomics by eliminating redundant menu items and consolidating user account settings into the bottom identity drawer.
AI CONTRIBUTION:	Refactored <code>Sidebar.jsx</code> , removed top-level profile links, and added an interactive popover menu to the user avatar drawer with logout triggers.
DEVELOPER CONTRIBUTION:	Evaluated visual layout on laptop and mobile screen widths, approved the consolidated navigation hierarchy, and confirmed reduced visual clutter.

Prompt 05 — Student Dashboard Architecture & Dynamic Targeted Mastery Cards

ADAPTIVE DASHBOARD

"Redesign `DashboardPage.jsx` into an action-oriented study command center that prioritizes active recall and dynamic learning recommendations. Requirements: Retire static onboarding checklists in favor of dynamic learning telemetry. Position a 'Targeted Mastery' recommendation card directly adjacent to 'Continue Studying', compute the user's weakest concept area in real-time with mastery percentage, and provide one-click 'Ask Tutor' and 'Practice Quiz' launch buttons."

ENGINEERING OBJECTIVE:	Transform the student dashboard from static onboarding checklists into an action-oriented study hub featuring dynamic AI-driven Targeted Mastery recommendations.
AI CONTRIBUTION:	Redesigned <code>DashboardPage.jsx</code> CSS grid layout, removing static cards and placing the Targeted Mastery card directly beside "Continue Studying".
DEVELOPER CONTRIBUTION:	Verified dynamic card rendering when concept mastery scores updated after quiz submissions and validated responsive breakpoints on desktop and mobile.

C. Backend Engineering

BACKEND

Prompt 06 — Production Role-Based Access Control (RBAC) & Filter Chain Enforcement

SECURITY ARCHITECTURE

"Architect and enforce strict Role-Based Access Control (RBAC) in Spring Boot 3 using Spring Security 6.4 and stateless JWTs. Requirements: Define explicit role boundaries between standard students (`ROLE_USER`) and administrators (`ROLE_ADMIN`). Ensure all new registrants default to `ROLE_USER` with zero privilege escalation paths, configure `SecurityFilterChain` authorization rules, and extract authorities inside `JwtAuthenticationFilter`."

ENGINEERING OBJECTIVE:	Eliminate privilege escalation vulnerabilities by establishing strict role boundaries between regular students (<code>ROLE_USER</code>) and system administrators (<code>ROLE_ADMIN</code>).
AI CONTRIBUTION:	Updated Spring Security filter chain configurations, refactored <code>User</code> entity role initialization to assign <code>ROLE_USER</code> by default, and secured API routes using method-level security.
DEVELOPER CONTRIBUTION:	Created separate test accounts (<code>ROLE_USER</code> vs <code>ROLE_ADMIN</code>), verified that non-admin tokens received HTTP 403 Forbidden on administrative endpoints, and confirmed database role persistence.

Prompt 07 — Securing Administrative Telemetry Endpoints with Method-Level Security

API AUTHORIZATION

"Harden the administrative monitoring endpoints in `AdminController.java` to prevent unauthorized telemetry harvesting. Requirements: Annotate all administrative routes (`/api/admin/**`) with `@PreAuthorize("hasRole('ADMIN')")`, update `GlobalExceptionHandler` to return RFC-7807 problem details with HTTP 403 Forbidden, and expose an authenticated authority check so the frontend conditionally renders the Admin navigation tab."

ENGINEERING OBJECTIVE:	Prevent information leakage and unauthorized UI exposure by enforcing backend <code>@PreAuthorize</code> guards on telemetry APIs and hiding admin navigation elements from non-admin principals.
AI CONTRIBUTION:	Added <code>@PreAuthorize("hasRole('ADMIN')")</code> annotations across <code>AdminController.java</code> , updated JWT claims parsing to extract authorities, and conditionally rendered the Admin UI tab in React.
DEVELOPER CONTRIBUTION:	Tested cross-account authentication states, verified that standard student accounts no longer see administrative tabs, and validated zero telemetry exposure in network inspector logs.

D. Database & Data Layer

DATABASE

Prompt 08 — Neon PostgreSQL 16 Provisioning, Flyway Migrations & pgvector Activation

VECTOR PERSISTENCE

"Configure a serverless PostgreSQL 16 persistence layer on Neon with connection pooling and high-dimensional vector search support. Requirements: Write a Flyway migration (`V1__init.sql`) executing `CREATE EXTENSION IF NOT EXISTS vector;`, define the `embedding` column in `document_chunks` as `vector(3072)` matching Gemini embeddings, and configure HikariCP in `application.yml` with `sslmode=require`."

ENGINEERING OBJECTIVE:	Provision managed relational and vector persistence with automated schema version control and connection pooling.
AI CONTRIBUTION:	Authored Flyway migration <code>V1__init.sql</code> with <code>CREATE EXTENSION IF NOT EXISTS vector;</code> , configured HikariCP connection pooling with <code>sslmode=require</code> in <code>application.yml</code> , and defined JPA entity mappings.
DEVELOPER CONTRIBUTION:	Executed Neon CLI linkage, configured Neon connection secrets in Render dashboard, monitored Flyway migration execution during boot, and inspected schema tables in Neon console.

Prompt 09 — Foreign Key Cascade Referential Integrity & Transactional Data Isolation

DATA INTEGRITY

"Establish comprehensive referential integrity and safe cascade deletion across the relational and vector data model. Requirements: Ensure deleting a `User` cascades down to delete their spaces, projects, study materials, vector chunks, chat sessions, and quiz attempts without orphaned rows. Write safe, parameterized SQL cleanup queries with strict transaction management to verify cascade behavior."

ENGINEERING OBJECTIVE:	Safely delete test user records from the production database while verifying referential integrity and cascade deletion behavior across spaces, projects, documents, chunks, and chat sessions.
AI CONTRIBUTION:	Formulated safe, transactional SQL deletion scripts ordering child entity cleanup (<code>user_roles</code> , <code>document_chunks</code> , <code>study_materials</code> , <code>chat_sessions</code> , <code>projects</code> , <code>spaces</code> , <code>users</code>) respecting foreign key constraints.
DEVELOPER CONTRIBUTION:	Executed queries in Neon SQL console, verified that deleting the user cleanly removed all associated vector embeddings and materials without leaving orphaned rows, and re-tested registration.

E. AI / LLM Engineering

AI / LLM

Prompt 10 — Dynamic Cognitive Mastery Scoring & Adaptive Recommendation Engine

COGNITIVE SCORING

"Design and implement a dynamic concept mastery calculation algorithm in `AnalyticsService.java` that computes continuous learning mastery scores (0%–100%) from real assessment data.

Requirements: Eliminate static mock percentages. Score each concept based on weighted parameters: quiz accuracy (60%), question difficulty weighting (20%), and recency/frequency of Socratic tutor inquiries (20%). Categorize into 'Mastered' (>=80%), 'Improving' (60-79%), and 'Needs Attention' (<60%), and expose the weakest concept to the dashboard."

ENGINEERING OBJECTIVE:

Replace arbitrary mock mastery percentages with a dynamic, data-driven concept mastery calculation reflecting genuine student quiz accuracy and study interaction depth.

AI CONTRIBUTION:

Designed and coded an adaptive calculation algorithm in `AnalyticsService.java` and `WorkspaceService.java` computing mastery scores (0%–100%) based on real quiz attempt scores, evaluation feedback, and study frequency.

DEVELOPER CONTRIBUTION:

Validated the updated scoring by completing quizzes with known scores (100% vs 50%), verifying that mastery percentages accurately reflected assessment performance without hardcoded defaults.

Prompt 11 — Grounded RAG Pipeline Architecture & Empty Materials Precondition Gate

RAG & HALLUCINATION DEFENSE

"Architect the Retrieval-Augmented Generation (RAG) pipeline for the AiProf tutor and quiz generation services using Google Gemini Flash Lite and pgvector.

Requirements: Ingest documents into 400-token chunks with 50-token overlap, embed via `gemini-embedding-001` (3072d), and execute cosine similarity search (`<=>`) with a 0.70 threshold. Enforce a strict prerequisite gate: if a project contains zero uploaded documents, block LLM invocation and prompt the user to upload course materials to prevent ungrounded hallucinations."

ENGINEERING OBJECTIVE:

Enforce strict grounding prerequisites preventing LLM hallucinations by blocking AI Tutor and Quiz generation until at least one syllabus or study document has been uploaded to the project.

AI CONTRIBUTION:

Implemented client-side and server-side validation gates (`materials.isEmpty()`) returning descriptive empty-state warnings and guiding students to upload course materials before invoking Gemini.

DEVELOPER CONTRIBUTION:

Tested empty projects and populated projects, verified that ungrounded prompt execution was blocked, and confirmed that grounding citations rendered accurately once documents were uploaded.

F. Authentication & Security

SECURITY

Prompt 12 — Google OAuth 2.0 Identity Services with Cryptographic Backend Verification

OAUTH 2.0 / SSO

"Implement enterprise-grade Google OAuth 2.0 authentication using Google Identity Services (GIS) on the frontend and cryptographic verification on the backend.

Requirements: Integrate `@react-oauth/google` on the client, transmit the Google ID token (JWT) to `POST /api/auth/google`, verify token signature and audience using `GoogleIdTokenVerifier` in `GoogleAuthService.java`, and issue our proprietary signed application JWT upon successful validation."

ENGINEERING OBJECTIVE:

Implement enterprise-grade Google OAuth 2.0 authentication to streamline student onboarding while ensuring cryptographic token validation on the backend.

AI CONTRIBUTION:

Integrated `@react-oauth/google` on the client, implemented `GoogleAuthService.java` using Google API Client libraries for ID token signature and audience verification, and handled account provisioning.

DEVELOPER CONTRIBUTION:

Configured Google Cloud Console OAuth credentials, registered authorized JavaScript origins and redirect URIs, configured client ID in Vercel environment variables, and validated end-to-end Google sign-in.

Prompt 13 — Time-Based 6-Digit Email OTP Verification State Machine

2FA / VERIFICATION GATE

"Implement a multi-step user registration workflow with mandatory 6-digit email OTP verification.

Requirements: Generate cryptographically secure 6-digit numeric codes using `SecureRandom`, store OTPs with a strict 5-minute time-to-live (TTL), rate-limit generation to 1 request per 60 seconds per email, and enforce an authentication state machine preventing account activation until OTP validation succeeds."

ENGINEERING OBJECTIVE:

Secure user registration against bot spam and unverified accounts by requiring a timed 6-digit one-time password delivered to the student's email before account activation.

AI CONTRIBUTION:

Built `OtpService.java` with cryptographically secure 6-digit generation, 5-minute expiration, in-memory caching, rate limiting, and an authentication state machine gating account activation.

DEVELOPER CONTRIBUTION:

Tested registration flow with real email accounts, verified expired OTP rejection, confirmed rate-limiting behavior, and approved the multi-step verification UX.

G. Debugging & Problem Solving

DEBUGGING

Prompt 14 — Diagnosing & Resolving Google OAuth Error 400: origin_mismatch in Production

OAUTH ORIGIN DEBUGGING

"Investigate and resolve Google OAuth Error 400: `origin_mismatch` occurring when users attempt Google Sign-In on the live Vercel deployment.

Requirements: Analyze Google OAuth 2.0 JavaScript origin policies and identify why the production preview URL was rejected. Provide configuration steps in Google Cloud Console to add authorized JavaScript origins for both the canonical production domain (`https://ai-prof-study-companion.vercel.app`) and local development (`http://localhost:5173`)."

ENGINEERING OBJECTIVE:

Diagnose and resolve a Google OAuth 2.0 policy rejection occurring on newly deployed Vercel domain URLs.

AI CONTRIBUTION:

Diagnosed that the active Vercel preview domain was not registered in Google Cloud Console's "Authorized JavaScript origins"; provided step-by-step resolution to add both canonical and preview domains.

DEVELOPER CONTRIBUTION:

Accessed Google Cloud Console, updated authorized web client origins with `https://ai-prof-study-companion.vercel.app` , and confirmed successful Google login in production.

Prompt 15 — Bypassing Cloud Egress SMTP Port Blocking via Resend HTTPS REST API

CLOUD NETWORK EGRESS

"Diagnose why email OTP verification codes fail to send in the Render cloud container environment despite working seamlessly in local development.

Requirements: Identify cloud hosting network egress limitations (Render blocks outbound SMTP ports 25, 465, 587 by default to prevent spam). Engineer an alternative delivery transport using Resend's HTTPS REST API over port 443 in `ResendEmailService.java` with automatic fallback to standard SMTP, and verify instant delivery."

ENGINEERING OBJECTIVE:

Resolve production email delivery failure caused by cloud hosting firewall restrictions on outbound SMTP ports (25, 465, 587).

AI CONTRIBUTION:

Identified that Render blocks outbound SMTP by default; engineered `ResendEmailService.java` using Resend's HTTPS REST API over port 443 with automatic fallback to standard SMTP.

DEVELOPER CONTRIBUTION:

Created a Resend account, provisioned an API key, configured `RESEND_API_KEY` on Render, and verified instantaneous OTP delivery to student inboxes.

Prompt 16 — Resolving Minified React Error #310 (Violation of the Rules of Hooks)

REACT LIFECYCLE CRASH

"Debug an intermittent blank screen crash on the production dashboard triggering Minified React Error #310 ('Rendered more hooks than during the previous render').

Requirements: Inspect `DashboardPage.jsx` for conditional hook execution or early return statements placed prior to `useMemo` , `useCallback` , or `useEffect` declarations. Refactor component lifecycle so all hooks unconditionally execute at the top level in invariant order before any loading skeletons or error boundaries are returned."

ENGINEERING OBJECTIVE:

Fix a critical runtime blank-screen bug on `DashboardPage.jsx` caused by conditional hook execution order.

AI CONTRIBUTION:

Traced the error to an early `if (loading) return <Spinner />` placed before `useMemo` declarations; refactored hook declarations to the top level before conditional returns.

DEVELOPER CONTRIBUTION:

Rebuilt frontend bundle (`npm run build`), tested dashboard rendering across authenticated and unauthenticated states, and verified zero console warnings.

H. Testing & Validation

TESTING & VALIDATION

Prompt 17 — End-to-End RAG Vector Pipeline & Socratic Tutor Functional Testing

AI PIPELINE VALIDATION

"Design and execute an end-to-end functional test suite for the RAG ingestion, vector search, and AI tutoring pipeline. Scenarios: Ingest a multi-page technical PDF, verify semantic chunking boundaries, and generate 3072d embeddings. Execute vector similarity search via pgvector, verify cosine distance scoring, confirm that Socratic Tutor responses strictly cite source chunks, and generate a 5-question diagnostic quiz validating compliance with requested JSON schemas."

ENGINEERING OBJECTIVE:	Execute comprehensive end-to-end verification of document ingestion, vector embedding generation, pgvector retrieval, Socratic tutor responses, and quiz generation.
AI CONTRIBUTION:	Formulated test validation scenarios covering semantic chunking, embedding generation via <code>gemini-embedding-001</code> , vector distance thresholding, and JSON schema compliance for quizzes.
DEVELOPER CONTRIBUTION:	Uploaded sample academic course notes, initiated interactive tutor sessions, verified citation source accuracy, took AI-generated quizzes, and validated automated grading.

Prompt 18 — Automated Headless Browser Smoke Testing & Session Persistence Audit

BROWSER AUTOMATION

"Deploy an autonomous browser subagent to execute a full smoke test of the live production application at `https://ai-prof-study-companion.vercel.app`. Audit Steps: Navigate to `/login`, verify Google Sign-In button rendering, authenticate using test credentials, verify JWT session storage in `localStorage`, confirm automatic redirection to `/dashboard`, and monitor browser console logs for zero unhandled exceptions or CORS rejections."

ENGINEERING OBJECTIVE:	Validate live production environment stability, authentication session restoration, and responsive UI layout via automated browser DOM inspection.
AI CONTRIBUTION:	Deployed an autonomous browser subagent to navigate to the production deployment URL, inspect DOM element hierarchies, evaluate JavaScript console error logs, and capture visual status screenshots.
DEVELOPER CONTRIBUTION:	Reviewed browser session recordings and screenshots, confirmed correct token restoration from <code>localStorage</code> , and approved production release readiness.

I. Deployment & Configuration

DEPLOYMENT & CONFIG

Prompt 19 — Production Multi-Domain OAuth & CORS Synchronization

DOMAIN SYNCHRONIZATION

"Configure environment variables and CORS policies across Vercel and Render following custom domain migration. Requirements: Update Spring Boot `CorsConfiguration` to allow requests from the canonical production domain (`https://ai-prof-study-companion.vercel.app`) with full credentials, headers, and standard HTTP methods. Configure `VITE_API_URL` and `VITE_GOOGLE_CLIENT_ID` in Vercel settings, and verify preflight HTTP OPTIONS requests succeed with HTTP 200."

ENGINEERING OBJECTIVE:	Synchronize Google OAuth credentials, Vercel frontend environment variables, and Spring Boot backend CORS configurations after assigning a production domain alias.
AI CONTRIBUTION:	Provided an exact audit of required configuration updates across <code>VITE_GOOGLE_CLIENT_ID</code> , Google Cloud Console Authorized Origins, and Spring Security <code>allowedOrigins</code> patterns.
DEVELOPER CONTRIBUTION:	Updated client environment settings, verified backend CORS filters accepted requests from the new domain alias, and tested Google Sign-In across browsers.

Prompt 20 — Disabling Vercel Deployment Protection for Public Evaluator Access

PUBLIC ACCESS BYPASS

"Configure Vercel project security settings to ensure the deployed platform is publicly accessible to external evaluators without requiring Vercel team authentication or login credentials. Requirements: Identify why external visitors encounter a 'Request Access' or Vercel SSO authentication wall. Provide instructions to disable 'Vercel Authentication' under Project Settings -> Deployment Protection, and verify external access in incognito windows."

ENGINEERING OBJECTIVE:	Eliminate Vercel's automated authentication barrier (SSO / Deployment Protection) to ensure evaluators can freely access and interact with the live application without a team invite.
AI CONTRIBUTION:	Identified that Vercel's "Deployment Protection" was intercepting requests with an access request modal; provided exact settings navigation path (<code>Settings -> Deployment Protection -> Vercel Authentication</code>) to disable it.
DEVELOPER CONTRIBUTION:	Configured Vercel project settings to disable deployment protection, verified public accessibility in private/incognito browser windows, and confirmed external usability.

J. Documentation

DOCUMENTATION

Prompt 21 — Production-Grade Repository Packaging, README & Secret Sanitization

REPOSITORY HYGIENE

"Prepare the entire repository for official evaluator review and submission.
Requirements: Write a comprehensive, publication-grade root `README.md` featuring project badges, architecture diagrams, core feature documentation, local quickstart steps, database migration commands, and API references. Perform an exhaustive security audit across git history and active files: purge all live API keys, database credentials, and secrets, replacing them with `.env.example` templates. Verify local build reproducibility, and strictly preserve application runtime code."

ENGINEERING
OBJECTIVE:

AI CONTRIBUTION:

DEVELOPER
CONTRIBUTION:

Prepare the entire repository for official evaluator review by structuring a comprehensive README, verifying local clone-and-run workflows, sanitizing secrets, and safeguarding working application code.

Crafted production-grade `README.md`, replaced hardcoded credentials with `.env.example` templates, sanitized `application.yml`, verified local build scripts, and drafted setup documentation.

Reviewed all sanitized configuration files, tested local build commands (`npm run build`, `./mvnw test-compile`), committed the repository, and pushed the clean public codebase to GitHub.

Prompt 22 — Publication-Grade System Architecture Technical Documentation

ARCHITECTURE DOCUMENTATION

"Author and compile an exhaustive System Architecture Documentation PDF (`AiProf_Architecture_Documentation.pdf`) for the AiProf platform.
Requirements: Document the decoupled three-tier topology, request-response lifecycles, complete ASCII flowcharts, entity-relationship database schemas, Spring Security filter chains, pgvector similarity query mechanics, and Render Dockerfile / Vercel cloud deployment architecture using headless Chrome PDF compilation."

ENGINEERING
OBJECTIVE:

AI CONTRIBUTION:

DEVELOPER
CONTRIBUTION:

Author an exhaustive technical reference document detailing system topology, data models, and architectural decisions.

Structured, authored, and compiled `AiProf_Architecture_Documentation.pdf` utilizing Chrome headless printing with professional diagrams and tables.

Reviewed technical accuracy, verified database schema representations and API endpoint mappings, and approved the document for intern submission.

Prompt 23 — Dual-Category AI Tools & Usage Technical Transparency Report

AI USAGE TRANSPARENCY

"Author and compile a transparent AI Tools & Usage Documentation PDF (`AiProf_AI_Tools_and_Usage_Documentation.pdf`) for project evaluation.
Requirements: Clearly separate Category A (AI used by the developer to build the software: Antigravity IDE, agent subagents, browser tools) from Category B (AI models embedded inside the product: Gemini Flash Lite, gemini-embedding-001, pgvector). Document prompt engineering methodologies, token efficiency, model costs, response latencies, and human oversight guardrails."

ENGINEERING
OBJECTIVE:

AI CONTRIBUTION:

DEVELOPER
CONTRIBUTION:

Provide a transparent, audited technical report detailing developer AI assistance contrasted with the production application's AI capabilities.

Authored and compiled `AiProf_AI_Tools_and_Usage_Documentation.pdf`, establishing clear categorization between development assistance and application features.

Verified transparency reporting, confirmed accurate model specifications, and approved the document for intern submission.

Engineering Prompt Patterns Demonstrated

METHODOLOGY

Analysis of Development Methodologies Across Full-Stack AI Engineering

Reviewing the prompts across the AiProf development lifecycle reveals ten distinct engineering prompt patterns. Rather than using AI as a basic chatbot or unguided code generator, the developer employed AI as an interactive, pair-programming engineering collaborator bounded by technical constraints, live verification, and architectural discipline.

1. Requirements → Implementation

Translating business and internship rubric requirements directly into working full-stack features (e.g., Socratic tutoring, concept mastery calculation, interactive quiz generation) without unnecessary intermediate boilerplate.

2. Architecture → Implementation

Structuring decoupled multi-tier systems (React 19 SPA + Spring Boot 3 + Neon PostgreSQL pgvector) and guiding the incremental step-by-step rollout across cloud infrastructure.

3. Feature Refinement & UX Iteration

Iteratively optimizing interface ergonomics based on real user testing, such as relocating profile settings into the user avatar drawer and elevating Targeted Mastery cards on the student dashboard.

4. Constraint-Driven Development

Enforcing strict operational boundaries on the AI assistant, prominently demonstrated by commands like "DONT CHANGE THE Application as it is perfect now" to prevent accidental regression during documentation.

5. Error-Driven Debugging & Root Cause Isolation

Providing exact runtime stack traces (`Minified React Error #310` , `Error 400: origin_mismatch`) and symptoms (`"otp is not coming"`), enabling immediate root-cause isolation and minimal diff patches.

6. Security Hardening & Zero-Trust Verification

Actively auditing role boundaries (`ROLE_USER` vs `ROLE_ADMIN`), guarding telemetry endpoints with `@PreAuthorize` , and implementing two-factor email OTP and Google OAuth 2.0 verification.

7. AI/RAG Pipeline Design & Hallucination Defense

Enforcing strict prerequisite gates (blocking tutor and quiz requests when document count is zero) and calibrating cosine similarity thresholds to guarantee grounded, cited LLM responses.

8. Autonomous Testing & Live Browser Validation

Deploying autonomous browser subagents and terminal compilation tests to inspect live production URLs, verify local storage JWT restoration, and confirm zero console warnings.

9. Production Deployment & Cloud Sync

Managing multi-cloud environment variables, coordinating OAuth authorized origins across preview and production URLs, and bypassing platform-level SSO deployment gates for public access.

10. Submission Packaging & Secret Sanitization

Preparing clean, evaluator-ready public repositories with thorough documentation, reproducible local setup instructions, and zero committed production credentials.

Representative Engineering Prompt Patterns (Reference Only)

EXAMPLE PATTERNS

Notice: Reference Architecture Prompts — Benchmark Standards

The following prompt templates illustrate mature engineering prompt formulations recommended for enterprise full-stack AI development. They are provided as reference benchmarks to demonstrate the prompt structure used by experienced AI engineers.

Representative Architecture Review Prompt (Senior Engineering Formulation)

"Review the current three-tier architecture (React SPA + Spring Boot + Neon PostgreSQL pgvector) and identify coupling, security, scalability, and deployment risks. Propose concrete refactoring steps that preserve our zero-breaking-change rule, minimize external cloud costs, and satisfy the internship project specifications."

Representative RAG Optimization Prompt (Senior Engineering Formulation)

"Analyze the document chunking and vector retrieval pipeline in DocumentChunkService.java. Compare recursive text splitting with semantic heading-aware chunking for academic textbook ingestion. Implement pgvector HNSW indexing on vector(3072) and benchmark cosine distance query latency under simulated concurrent student query loads."

Submission Verification & Deliverables Summary

DELIVERABLES

SUBMISSION DOCUMENT	FILENAME	FORMAT & ENGINE	STATUS & VERIFICATION
Artifact I: System Architecture	AiProf_Architecture_Documentation.pdf	PDF (Headless Chrome)	Completed • Published to GitHub repository root
Artifact II: AI Tools & Usage	AiProf_AI_Tools_and_Usage_Documentation.pdf	PDF (Headless Chrome)	Completed • Published to GitHub repository root
Artifact III: Prompts Used	AiProf_AI_Prompts_Used_During_Development.pdf	PDF (Headless Chrome)	Completed • 23 Prompts across 10 Domains + Synthesis
Production Application	ai-prof-study-companion.vercel.app	Live Web Platform	Publicly accessible • Zero SSO gate • Fully verified

AiProf Study Companion • AI Prompts Used During Development • Prepared for Full Stack AI Engineer Intern Project Submission

Senior full-stack AI engineering prompts formulated for production development • Confirmed and validated by Sathwik Bodakunta